Pretrained Language Models

MLP LAB KyungTae Lim

Contents

01 Motivating word meaning and context

02 Pretraining methods

03 Pretraining on Encoder

04 Pretraining on Decoder

05 Applying BERT and GPT

01. Motivating word meaning and context

Word Representation

- How to represent symbolic characters as computable numbers?
- Can we just represent numbers? What about people? Consider the word "tree".

• signifier (symbol) ⟺ signified (idea or thing)

= denotational semantics

tree ⟺ { 🌳 , 🌲 , 🌴 ,…}

- When AI express a word, it has to imply the semantics of the word, right?

Ref: Starford cs224n

Word Representation

- Semantics: let similar words be represented by similar vector values

Example: in web search, if a user searches for "Seattle motel", we would like to match documents containing "Seattle hotel"

motel = [0 0 0 0 0 0 0 0 0 0 1 0 0 0 0]
hotel = [0 0 0 0 0 0 0 1 0 0 0 0 0 0 0]

These two vectors are orthogonal

There is no natural notion of **similarity** for one-hot vectors!

Solution:

motel = [0.34 -1.22 3.31]
hotel = [0.30 -1.01 3.31]

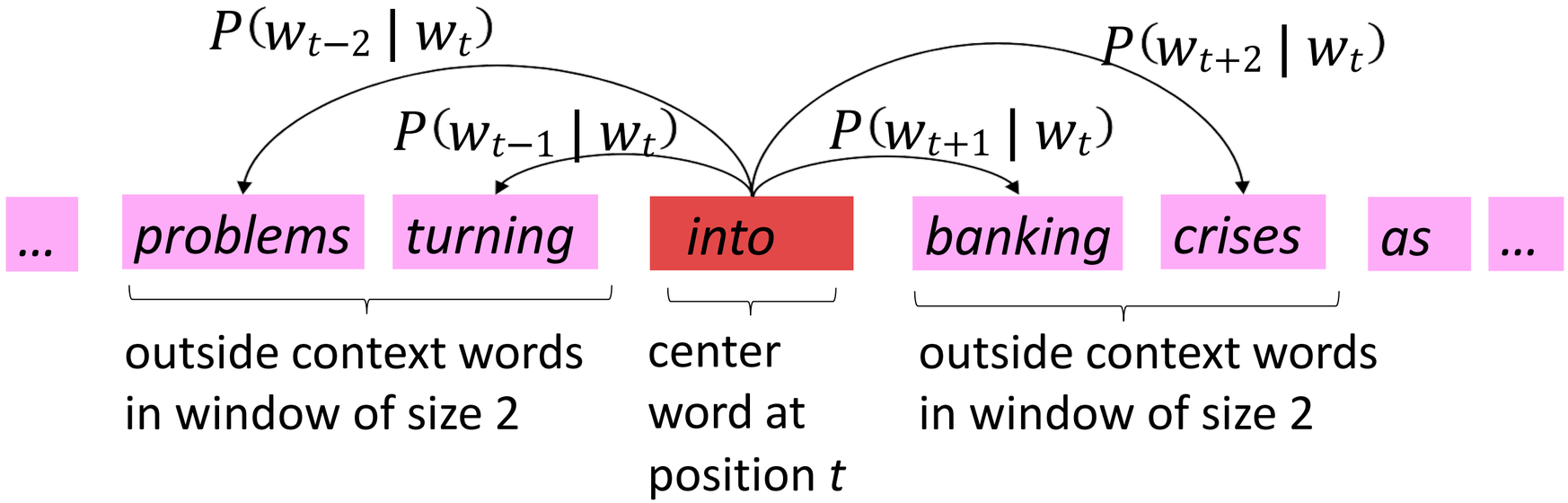
now "motel" and "hotel" is similar

Ref: Starford cs224n

Word2Vec Overview (Skipgram)

- Word2vec (Mikolov et al. 2013) is a framework for learning word vectors

Example windows and process for computing $P(w_{t+j} \mid w_t)$



Ref: Starford cs224n

Let just consider this sentence with word2vec embeddings:

*“I **record** the **record**”*

Hmm... but how do we distinguish between the two "record" in the sentence?

The two instances of ***record*** mean different things.

Let just consider this sentence with word2vec embeddings:

*“I **record** the **record**”*

Hmm… but how do we distinguish between the two "record" in the sentence?

Aha, passing the RNN through will change the representation of the two records while taking into account the surrounding context!

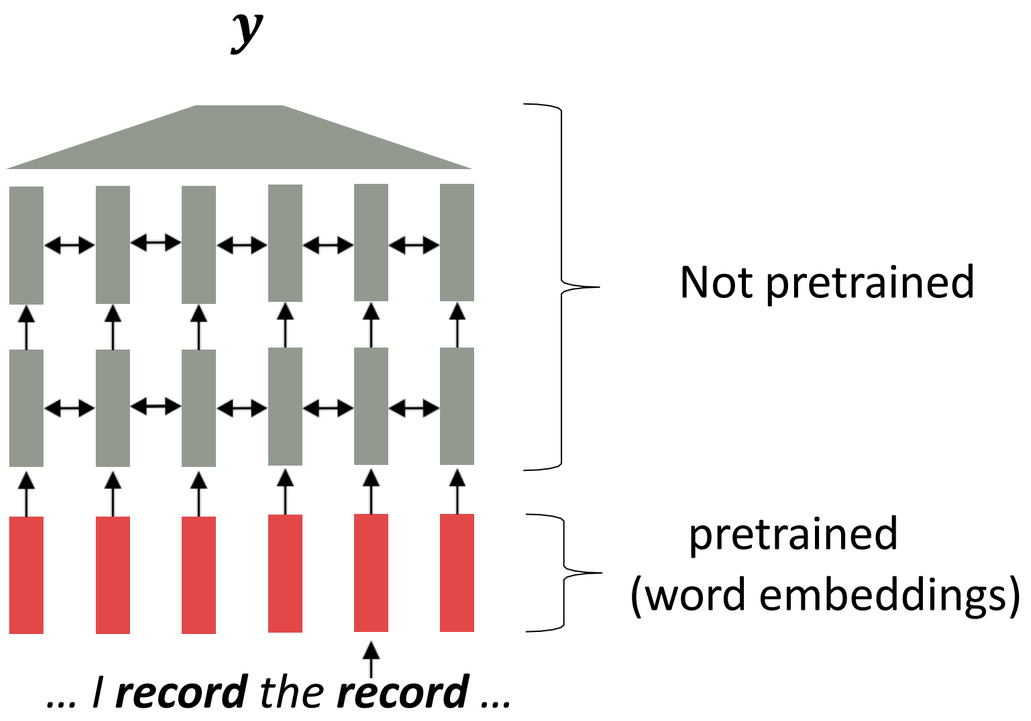
Where we were: pretrained word embeddings

Circa 2017:

- Start with pretrained word embeddings (no context!)
- Learn how to incorporate context in an RNN or Transformer while training on the task.

Some issues to think about:

- The training data we have for our **downstream task** (like Sentiment ANA) must be sufficient to teach all contextual aspects of language. → language model
- Most of the parameters in our network are randomly initialized!



[Recall, *record* gets the same word embedding, no matter what sentence it shows up in]

Ref: Starford cs224n

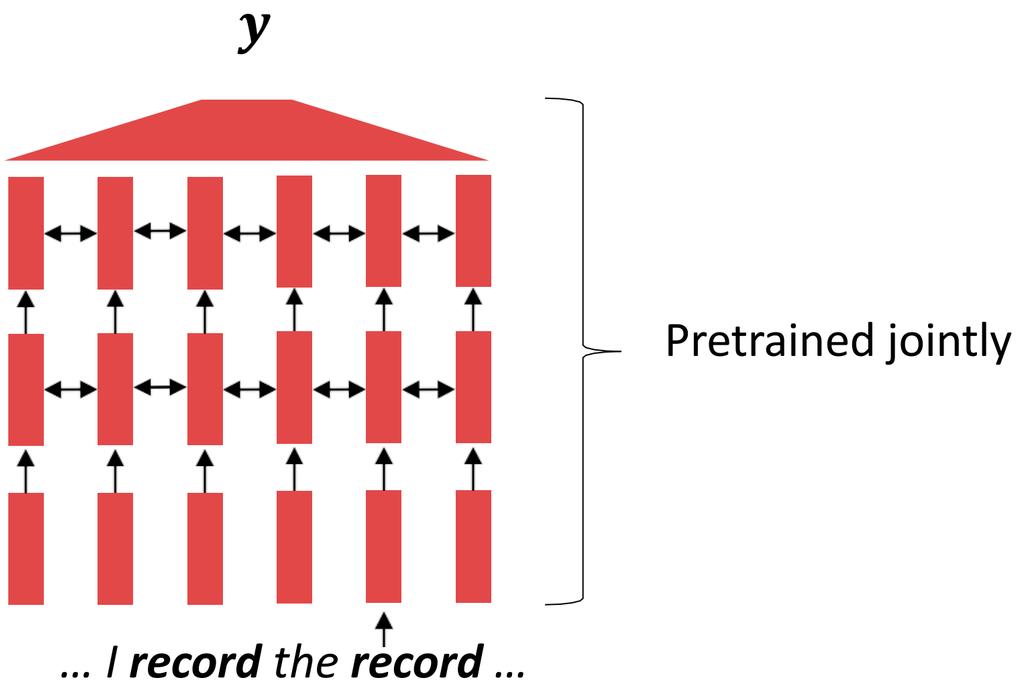
Where we're going: pretraining whole models

Pretraining in NLP: Teach all contextual aspects of language in advance using plaintext first

In modern NLP:

- All (or almost all) parameters in NLP networks are initialized via **pretraining**.
- Pretraining methods hide parts of the input from the model, and train the model to reconstruct those parts.

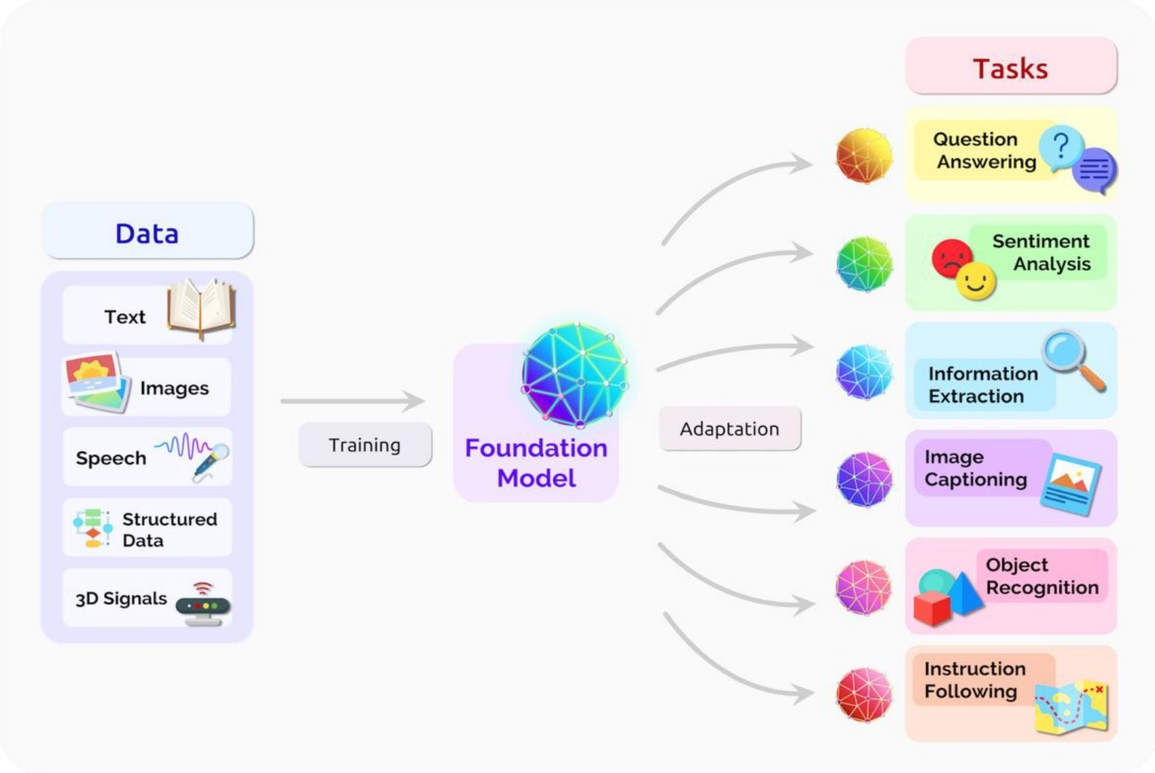
- This has been exceptionally effective at building strong:
  - **representations of language**
  - **parameter initializations** for strong NLP models.
  - **Probability distributions** over language that we can sample from



[This model has learned how to represent entire sentences through pretraining]

Ref: Starford cs224n

Pretraining methods

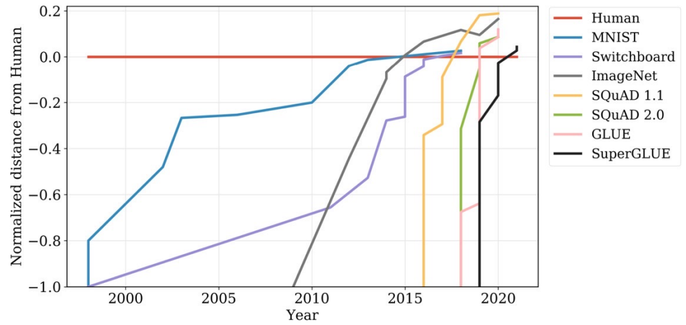


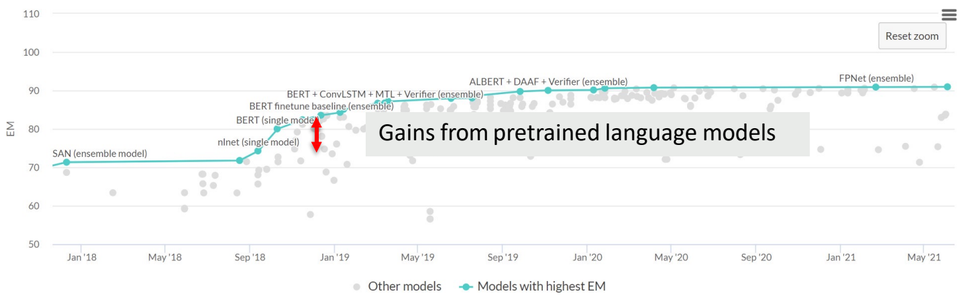
Key ideas in pretraining

- Make sure your model can process large-scale, diverse datasets
- Don’t use labeled data (otherwise you can’t scale!)
- Compute-aware scaling

Pretraining methods

- Pretraining in language models is the process of initially training a model on massive amounts of text data, enabling it to learn general language patterns and knowledge. This initial training helps the model quickly adapt to specific tasks later on.





Pretraining has had a major, tangible impact on how well NLP systems work

Pretraining methods

- To apply pretraining formally, we are using unsupervised-learning!! (why?)
  - Orders of magnitude difference in data size – there is a lot of high-quality text

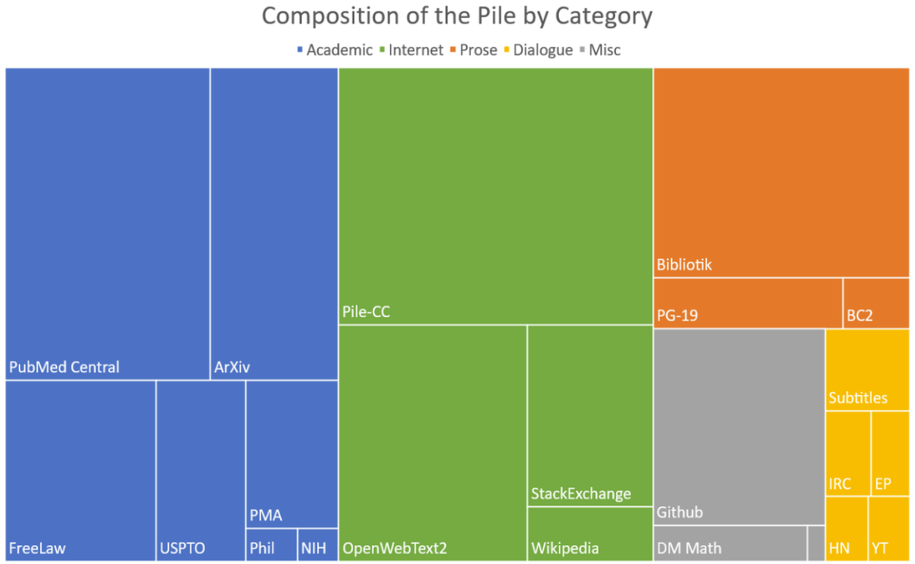
Dataset	Tokens (~0.75 words)
SQuAD 2.0 [Rajpukar+ 2018]	< 50 Million
DCLM-pool [Li+ 2024]	240 Trillion
Estimated ‘internet text’ [Villalobos 2024]	510T (indexed), 3100T (total)

A *10 million* times gap in QA to indexed internet

With this much data, we might make progress on even the hardest fill-in-the-blank tasks

Pretraining methods

- To apply pretraining formally, we are using unsupervised-learning!! (why?)
  - It's not just about the quantity, but also the incredible *diversity* of internet text data



[Gao+ 20]

Source	Doc Type	UTF-8 bytes *(GB)*	Documents *(millions)*	Unicode words *(billions)*	Llama tokens *(billions)*
Common Crawl	web pages	9,812	3,734	1,928	2,479
GitHub	code	1,043	210	260	411
Reddit	social media	339	377	72	89
Semantic Scholar	papers	268	38.8	50	70
Project Gutenberg	books	20.4	0.056	4.0	6.0
Wikipedia, Wikibooks	encyclopedic	16.2	6.2	3.7	4.3
Total		**11,519**	**4,367**	**2,318**	**3,059**

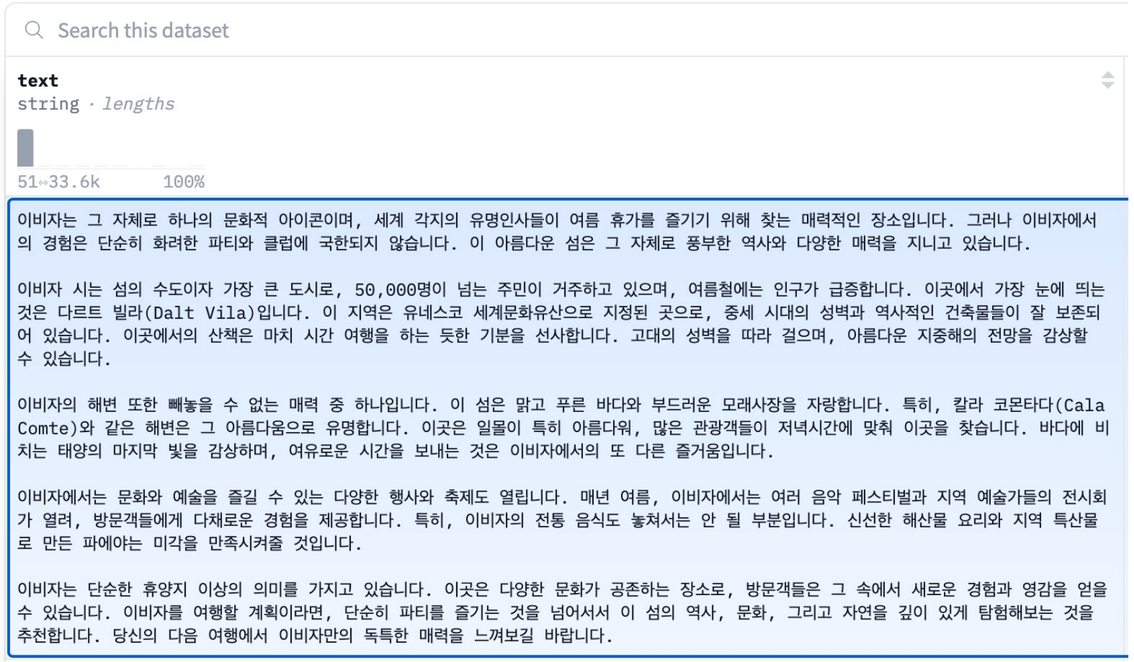
[Soldani+ 24]

This gives us some weak coverage over an enormous range of downstream tasks

Pretraining methods

- To apply pretraining formally, we are using unsupervised-learning!! (why?)
  - It’s not just about the quantity, but also the incredible *diversity* of internet text data

- Bizarro Wonder Woman is a bizarro version of Wonder Woman.\n\nWhen Bizarro III found himself infused with radiation from a blue sun, he developed the ability to replicate himself as well as create other \"Bizarro\" lifeforms based upon likenesses of people from Earth. He used this power to populate a cube-shaped planetoid dubbed Bizarro World within the blue sun star system. One of the many duplicates that he created was a Bizarro version of Wonder Woman. Bizarro Wonder Woman, working alongside her Bizarro confederates Batman, Flash, Green Lantern and Hawkgirl, sought to save Bizarro from Bizarro Doomsday by dropping their hyperbolic headquarters on top of him.\n\nAs opposed to her counterpart, Bizarro Wonder Woman uses a lasso that causes those ensnared to tell lies.\ …



Pretraining methods

- To apply pretraining formally, we are using unsupervised-learning!! (why?)
  - But sometimes it makes issues about the fair use

Google swallows 11,000 novels to improve AI's conversation

As writers learn that tech giant has processed their work without permission, the Authors Guild condemns 'blatantly commercial use of expressive authorship'

'It doesn't harm the authors' ... Google's headquarters in Mountain View, California. Photograph: Marcio Jose Sanchez/AP

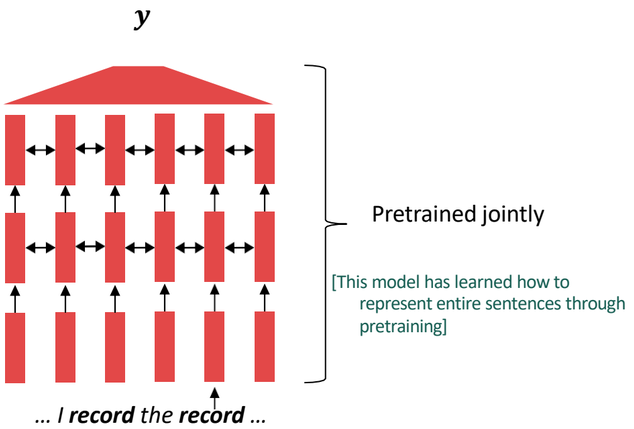
Arts and Humanities, Law, Regulation, and Policy, Machine Learning

Reexamining "Fair Use" in the Age of AI

Generative AI claims to produce new language and images, but when those ideas are based on copyrighted material, who gets the credit? A new paper from Stanford University looks for answers.

Jun 5, 2023 | Andrew Myers

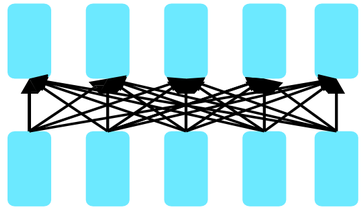
02. Pretraining Methods



So, how can we jointly pretrain between word embeddings and RNNs in a structure?

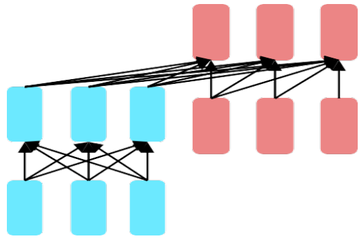
Pretraining for Three types of architectures

- Three popular neural net-based pretraining methods



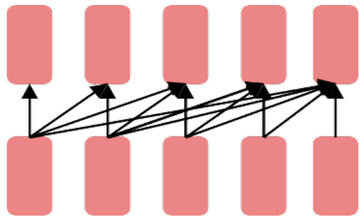
Encoders

- Gets bidirectional context – can condition on future!
- How do we train them to build strong representations?



Encoder-Decoders

- Good parts of decoders and encoders?
- What's the best way to pretrain them?

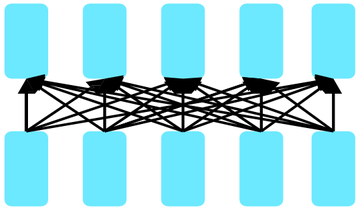


Decoders

- Language models! What we've seen so far.
- Nice to generate from; can't condition on future words

Pretraining for Three types of architectures

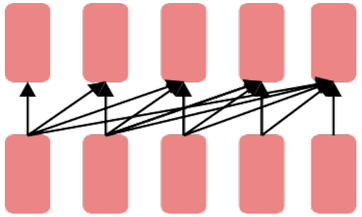
- The structure of Encoder VS Decoder



Encoders

- Gets bidirectional context – can condition on future!
- How do we train them to build strong representations?

Uh-oh, the encoder and decoder look different.



Decoders

- Language models! What we've seen so far.
- Nice to generate from; can't condition on future words

Pretraining for Three types of architectures

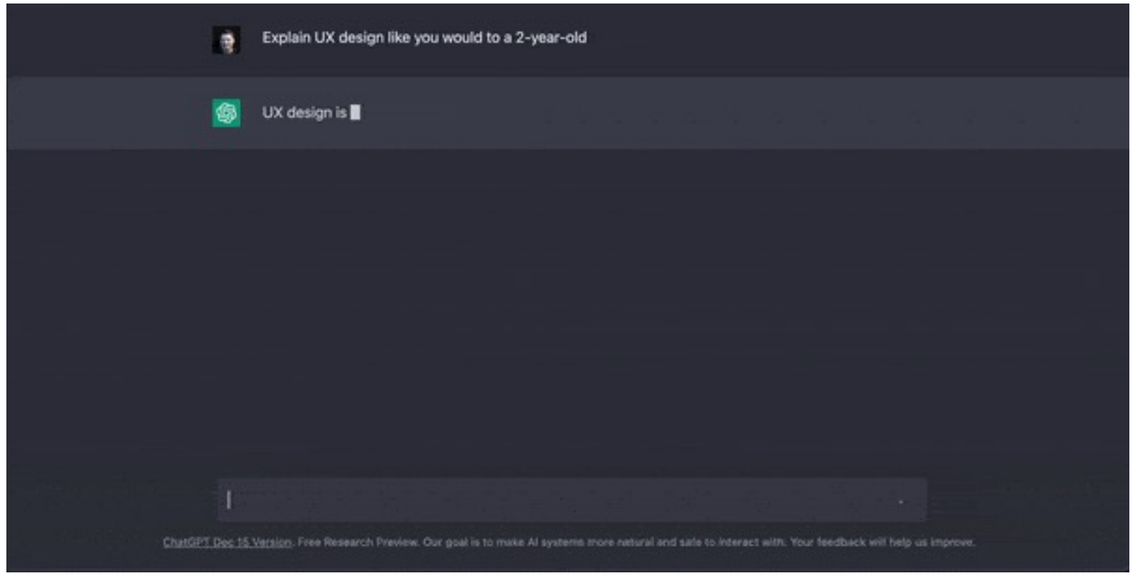
- The structure of Encoder VS Decoder

Encoder Input: Explain UX design like you would to a 2-year-old

Sentences are confirmed as you type

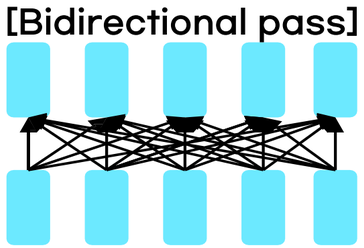
Decoder Input: UX design is ···

Sentences are finalized on output



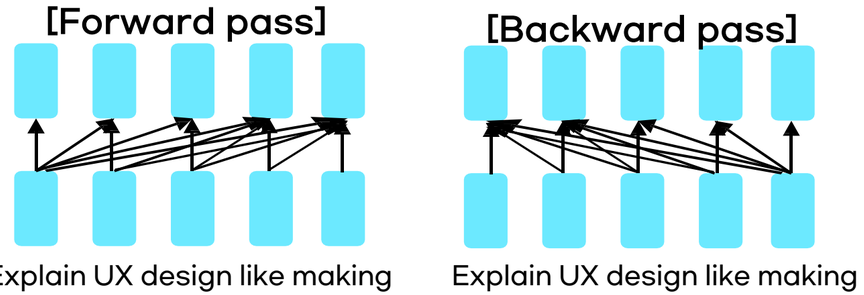
Pretraining for Three types of architectures

- The structure of Encoder VS Decoder



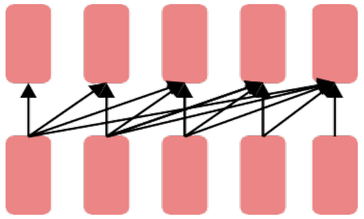
Encoder Input: Explain UX design like you would to a 2-year-old

Determined == Bidirectional == Auto Encoding



Decoder Input: UX design is ···

Confirmed at output == One-way operations only == Auto Regressive



Auto Regressive and Auto Encoding

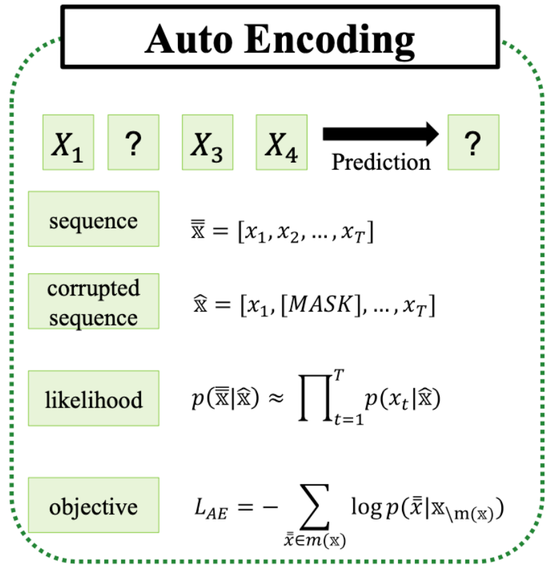
- The structure of Encoder VS Decoder

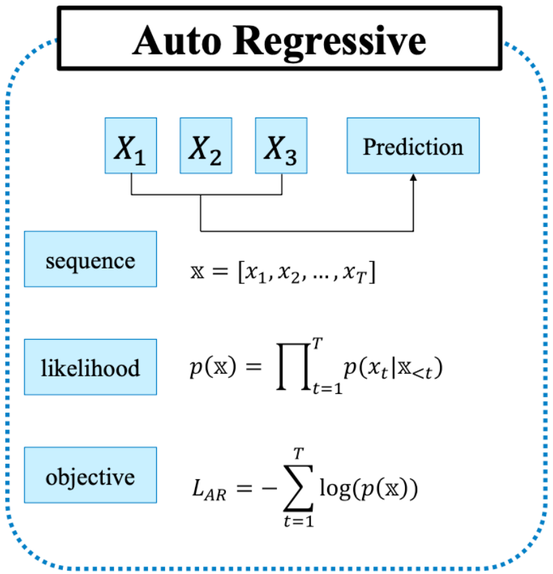
Encoder Input: Explain UX design like you would to a 2-year-old

Determined == Bidirectional == Auto Encoding

Decoder Input: UX design is …

Confirmed at output == One-way operations only == Auto Regressive





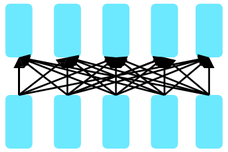
$m(\mathbb{x})$ and $\mathbb{x}_{\backslash m(\mathbb{x})}$ denote the masked words from $\mathbb{x}$ and the rest words respectively (MLM은 AE의 한 종류에 속함)

03. Pretraining on Encoder

Pretraining on Encoder

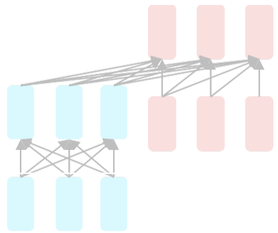
- Pretraining methods on Encoder with a focus on natural language understanding.

The neural architecture influences the type of pretraining, and natural use cases.



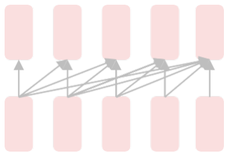
Encoders

- Gets bidirectional context – can condition on future!
- How do we train them to build strong representations?



Encoder-Decoders

- Good parts of decoders and encoders?
- What's the best way to pretrain them?



Decoders

- Language models! What we've seen so far.
- Nice to generate from; can't condition on future words

Pretraining on Encoder

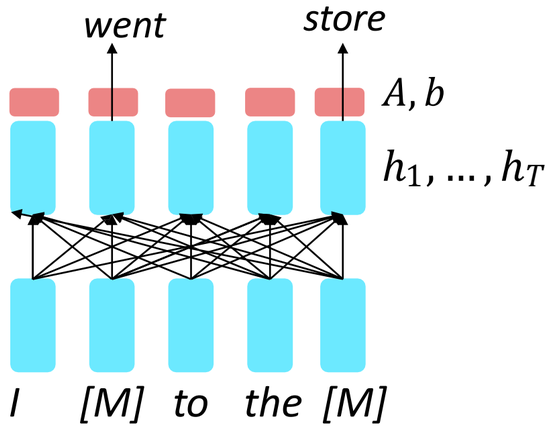
- Pretraining methods on Encoder with a focus on natural language understanding.

So far, we've looked at language model pretraining. But **encoders get bidirectional context,** so we can't do language modeling!

Idea: replace some fraction of words in the input with a special [MASK] token; predict these words.

$$h_1, \dots, h_T = \text{Encoder}(w_1, \dots, w_T)$$
$$y_i \sim Aw_i + b$$

Only add loss terms from words that are "masked out." If $\tilde{x}$ is the masked version of x, we're learning $p_\theta(x|\tilde{x})$Called **Masked LM**.



[Devlin et al., 2018]

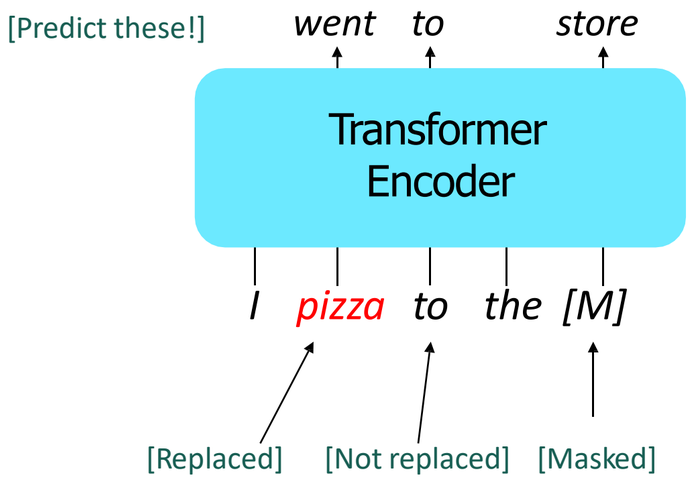
Pretraining on Encoder

- BERT: Bidirectional Encoder Representations from Transformers

Devlin et al., 2018 proposed the “Masked LM” objective and **released the weights of a pretrained Transformer**, a model they labeled BERT.

Some more details about Masked LM for BERT:

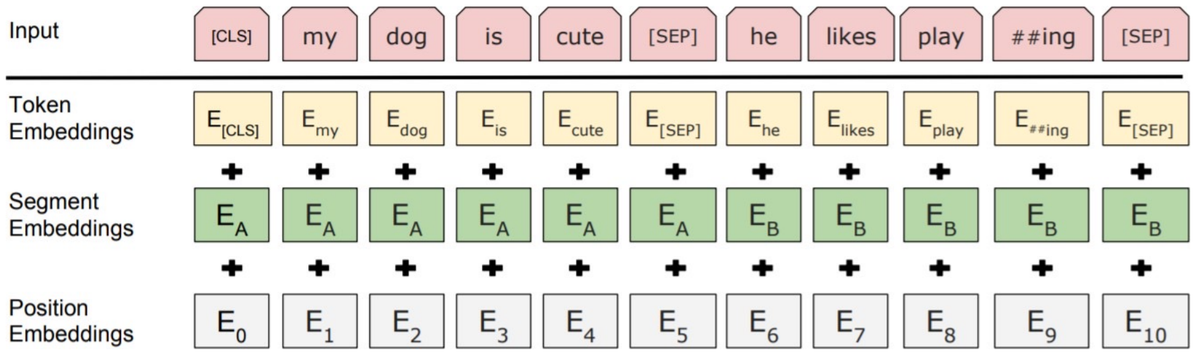
- Predict a random 15% of (sub)word tokens.
  - Replace input word with [MASK] 80% of the time
  - Replace input word with a random token 10% of the time
  - Leave input word unchanged 10% of the time (but still predict it!)
- Why? Doesn’t let the model get complacent and not build strong representations of non-masked words. (No masks are seen at fine-tuning time!)



[Devlin et al., 2018]

Pretraining on Encoder (Representative Models)

- BERT: Bidirectional Encoder Representations from Transformers
  - The pretraining input to BERT was two separate contiguous chunks of text:



  - BERT was trained to predict whether one chunk follows the other or is randomly sampled.
    - Later work has argued this “next sentence prediction” is not necessary.

Pretraining on Encoder (Representative Models)

- BERT: Bidirectional Encoder Representations from Transformers

BERT was massively popular and hugely versatile; finetuning BERT led to new state-of-the-art results on a broad range of tasks.

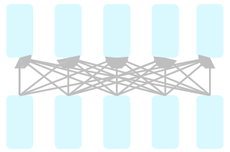
- **QQP:** Quora Question Pairs (detect paraphrase questions)
- **QNLI**: natural language inference over question answering data
- **SST-2**: sentiment analysis
- **CoLA**: corpus of linguistic acceptability (detect whether sentences are grammatical.)
- **STS-B**: semantic textual similarity
- **MRPC**: microsoft paraphrase corpus
- **RTE**: a small natural language inference corpus

System	MNLI-(m/mm) 392k	QQP 363k	QNLI 108k	SST-2 67k	CoLA 8.5k	STS-B 5.7k	MRPC 3.5k	RTE 2.5k	**Average** -
Pre-OpenAI SOTA	80.6/80.1	66.1	82.3	93.2	35.0	81.0	86.0	61.7	74.0
BiLSTM+ELMo+Attn	76.4/76.1	64.8	79.8	90.4	36.0	73.3	84.9	56.8	71.0
OpenAI GPT	82.1/81.4	70.3	87.4	91.3	45.4	80.0	82.3	56.0	75.1
$\text{BERT}_{\text{BASE}}$	84.6/83.4	71.2	90.5	93.5	52.1	85.8	88.9	66.4	79.6
$\text{BERT}_{\text{LARGE}}$	**86.7/85.9**	**72.1**	**92.7**	**94.9**	**60.5**	**86.5**	**89.3**	**70.1**	**82.1**

04. Pretraining on Decoder

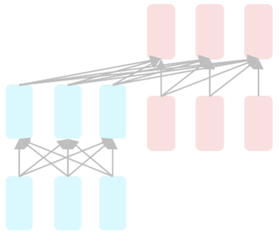
Pretraining on Decoder

- Pre-training based on the Decoder model with a focus on natural language generation



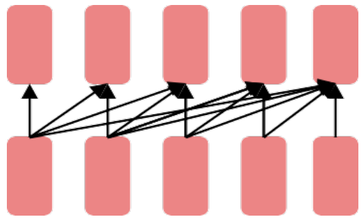
Encoders

- Gets bidirectional context – can condition on future!
- How do we train them to build strong representations?



Encoder-Decoders

- Good parts of decoders and encoders?
- What's the best way to pretrain them?



Decoders

- Language models! What we've seen so far.
- Nice to generate from; can't condition on future words
- All the biggest pretrained models are Decoders.

Pretraining on Decoder

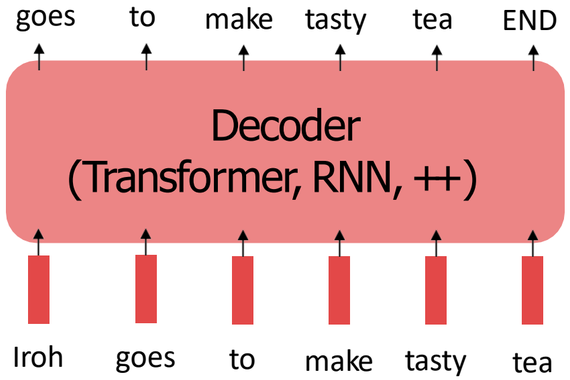
- Decoder's pre-training method: Save parameters after training language model

Recall the **language modeling** task:

- Model $p_\theta(w_t|w_{1:t-1})$, the probability distribution over words given their past contexts.
- There's lots of data for this! (In English.)

Pretraining through language modeling:

- Train a neural network to perform language modeling on a **large amount of text**.
- **Save the network parameters.**

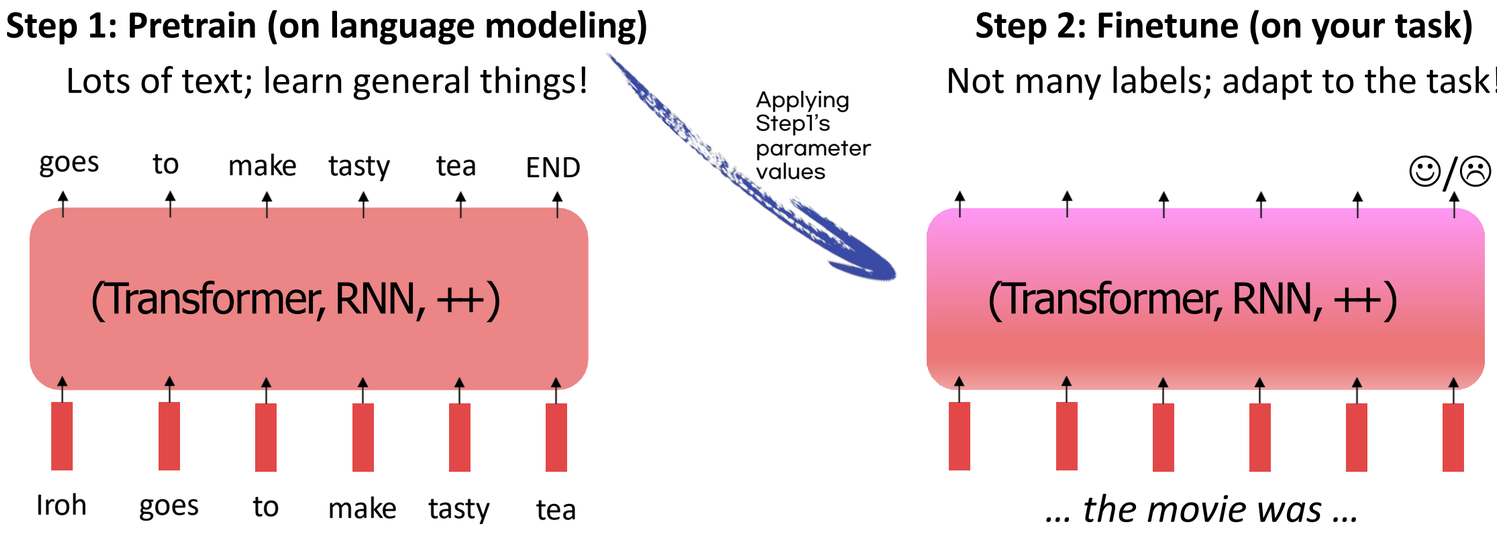


Pretraining on Decoder

- How to Use the Pretraining Model?: Finetuning

  - Use pre-trained parameters with Language Modeling as initial values when learning a specific task

Pretraining can improve NLP applications by serving as **parameter initialization**.



Pretraining on Decoder

- How to Use the Pretraining Model?: Finetuning

  - Use pre-trained parameters with Language Modeling as initial values when learning a specific task

  - This is like someone who has roller-skated for 10 years mastering figure skating in just 7 days.

Step 1: Pretrain (on language modeling)

Lots of text; learn **general** things!

Leverage initial values (experience)

Step 2: Finetune (on your task)

Not many labels; adapt to the task!

Pretraining on Decoder

- How to Use the Pretraining Model?: Finetuning

  - Use pre-trained parameters with Language Modeling as initial values when learning a specific task

    - Consider, provides parameters $\hat{\theta}$ by approximating $\min_{\theta} \mathcal{L}_{\text{pretrain}}(\theta)$.
      - (The pretraining loss.)
    - Then, finetuning approximates $\min_{\theta} \mathcal{L}_{\text{finetune}}(\theta)$, starting at $\hat{\theta}$.
      - (The finetuning loss)
    - The pretraining may matter because stochastic gradient descent sticks (relatively) close to $\hat{\theta}$ during finetuning.
      - So, maybe the finetuning local minima near $\hat{\theta}$ tend to generalize well!
      - And/or, maybe the gradients of finetuning loss near $\hat{\theta}$ propagate nicely!

Pretraining on Decoder

- The usage of pretrained model: Finetuning for Classification (e.g., Sentiment ana)

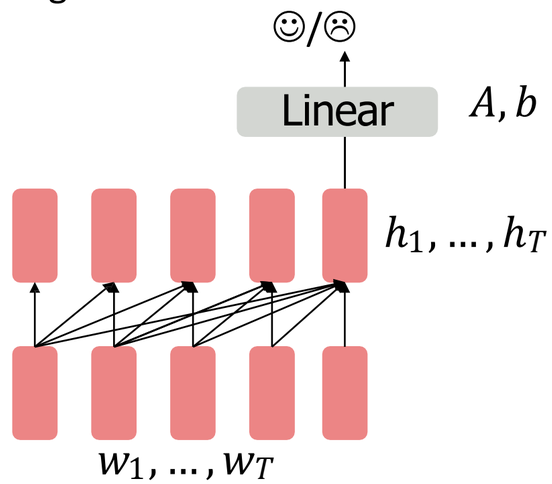
When using language model pretrained decoders, we can ignore that they were trained to model $p(w_t|w_{1:t-1})$.

We can finetune them by training a classifier on the last word's hidden state.

$$h_1, \ldots, h_T = \text{Decoder}(w_1, \ldots, w_T)$$
$$y \sim Ah_T + b$$

Where A and b are randomly initialized and specified by the downstream task.

Gradients backpropagate through the whole network.



[Note how the linear layer hasn't been pretrained and must be learned from scratch.]

Pretraining on Decoder

- The usage of pretrained model: Finetuning for Generation (e.g., Chatbot)

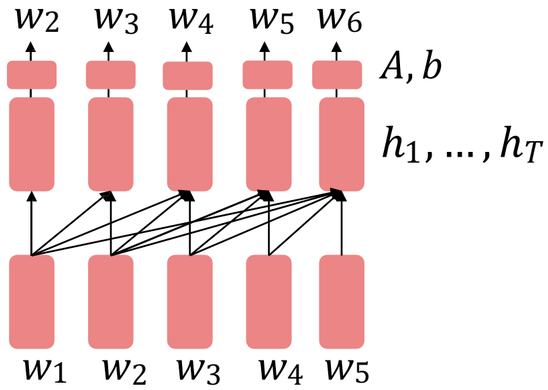
It's natural to pretrain decoders as language models and then use them as generators, finetuning their $p_\theta(w_t|w_{1:t-1})$!

This is helpful in tasks **where the output is a sequence** with a vocabulary like that at pretraining time!

- Dialogue (context=dialogue history)
- Summarization (context=document)

$$h_1, \ldots, h_T = \text{Decoder}(w_1, \ldots, w_T)$$
$$w_t \sim Ah_{t-1} + b$$

Where A, b were pretrained in the language model!



[Note how the linear layer has been pretrained.]

Pretraining on Decoder (Representative Models)

Generative Pretrained Transformer (GPT) [Radford et al., 2018]

2018's GPT was a big success in pretraining a decoder!

- Transformer decoder with 12 layers, 117M parameters.
- 768-dimensional hidden states, 3072-dimensional feed-forward hidden layers.
- Byte-pair encoding with 40,000 merges
- Trained on BooksCorpus: over 7000 unique books.
  - Contains long spans of contiguous text, for learning long-distance dependencies.
- The acronym "GPT" never showed up in the original paper; it could stand for "Generative PreTraining" or "Generative Pretrained Transformer"

Pretraining on Decoder (Representative Models)

Generative Pretrained Transformer (GPT) [Radford et al., 2018]

How do we format inputs to our decoder for **finetuning tasks?**

Natural Language Inference: Label pairs of sentences as *entailing/contradictory/neutral*

Premise: *The man is in the doorway*
Hypothesis: *The person is near the door* } **entailment**

Radford et al., 2018 evaluate on natural language inference.

Here's roughly how the input was formatted, as a sequence of tokens for the decoder.

[START] *The man is in the doorway* [DELIM] *The person is near the door* [EXTRACT]

The linear classifier is applied to the representation of the [EXTRACT] token.

Pretraining on Decoder (Representative Models)

Generative Pretrained Transformer (GPT) [Radford et al., 2018]

- GPT + finetuning method outperforms existing proposed methods for NLI problems!

Method	MNLI-m	MNLI-mm	SNLI	SciTail	QNLI	RTE
ESIM + ELMo [44] (5x)	-	-	89.3	-	-	-
CAFE [58] (5x)	80.2	79.0	89.3	-	-	-
Stochastic Answer Network [35] (3x)	80.6	80.1	-	-	-	-
CAFE [58]	78.7	77.9	88.5	83.3		
GenSen [64]	71.4	71.3	-	-	82.3	59.2
Multi-task BiLSTM + Attn [64]	72.2	72.1	-	-	82.1	**61.7**
Finetuned Transformer LM (ours)	**82.1**	**81.4**	**89.9**	**88.3**	**88.1**	56.0

Pretraining on Decoder (Representative Models)

Increasingly convincing generations (GPT2) [Radford et al., 2018]

- GPT2 generates human-like sentences for document summarization, translation, and more.

We mentioned how pretrained decoders can be used **in their capacities as language models.** **GPT-2,** a larger version (1.5B) of GPT trained on more data, was shown to produce relatively convincing samples of natural language.

Context (human-written): In a shocking finding, scientist discovered a herd of unicorns living in a remote, previously unexplored valley, in the Andes Mountains. Even more surprising to the researchers was the fact that the unicorns spoke perfect English.
GPT-2: The scientist named the population, after their distinctive horn, Ovid's Unicorn. These four-horned, silver-white unicorns were previously unknown to science. Now, after almost two centuries, the mystery of what sparked this odd phenomenon is finally solved. Dr. Jorge Pérez, an evolutionary biologist from the University of La Paz, and several companions, were exploring the Andes Mountains when they found a small valley, with no other animals or humans. Pérez noticed that the valley had what appeared to be a natural fountain, surrounded by two peaks of rock and silver snow.

Pretraining on Decoder (Representative Models)

GPT-3, In-context learning, and very large models

- GPT3 generates answers based on a few example commands without any finetuning.

So far, we've interacted with pretrained models in two ways:

- Sample from the distributions they define (maybe providing a prompt)
- Fine-tune them on a task we care about, and take their predictions.

Very large language models seem to perform some kind of learning **without gradient steps** simply from examples you provide within their contexts.

GPT-3 is the canonical example of this. The largest T5 model had 11 billion parameters.

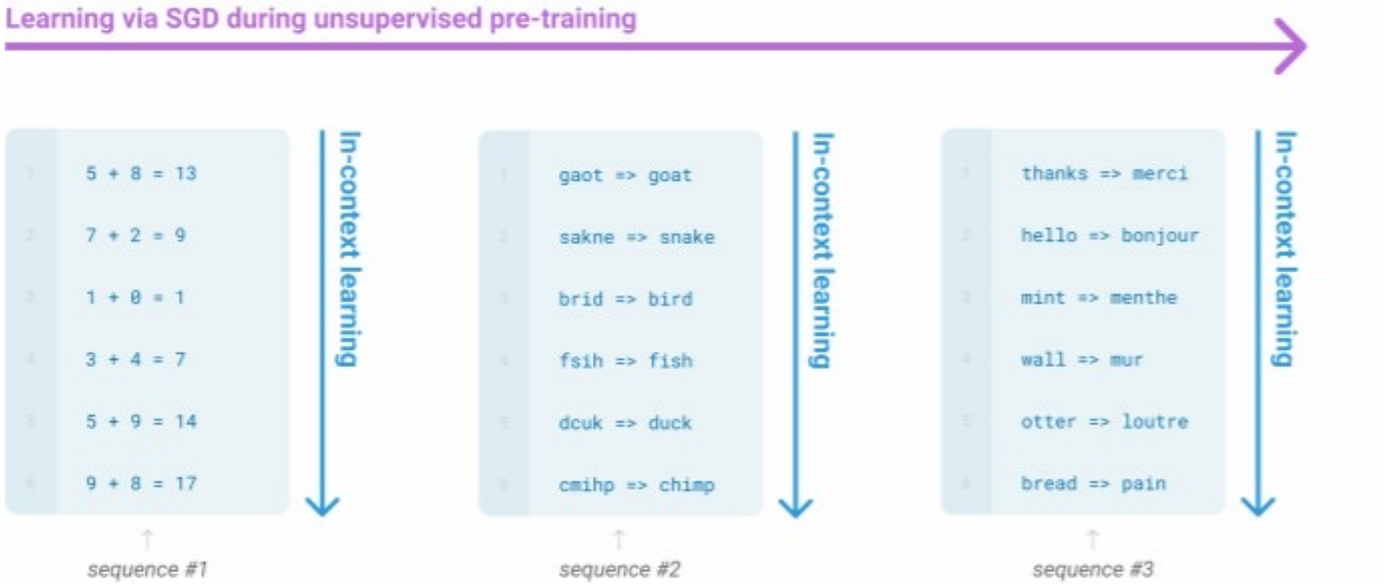
GPT-3 has 175 billion parameters.

Pretraining on Decoder (Representative Models)

GPT-3, In-context learning, and very large models

- GPT3 generates answers based on a few example commands without any finetuning.

Very large language models seem to perform some kind of learning **without gradient steps** simply from examples you provide within their contexts.



Pretraining on Decoder (Representative Models)

- ChatGPT, GPT, Gemini, Alpaca. Opening of the Warring States Period!

Stanford Alpaca

deepseek

Gemini

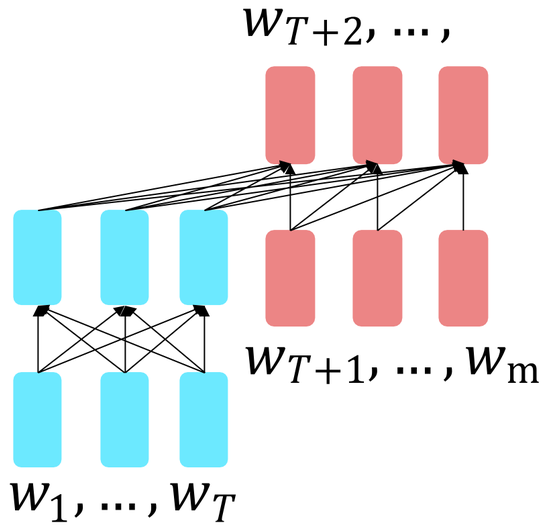
05. Pretraining on Encoder-Decoder

Pretraining on Encoder-Decoder

For **encoder-decoders**, we could do something like **language modeling**, but where a prefix of every input is provided to the encoder and is not predicted.

$$h_1, \ldots, h_T = \text{Encoder}(w_1, \ldots, w_T)$$
$$h_{T+1}, \ldots, h_2 = Decoder\,(w_{\text{t}+1}, \ldots, w_{\text{m}}, h_1, \ldots,)$$
$$h_T$$
$$y_i \sim Ah_i + b, i > T$$

The **encoder** portion benefits from bidirectional context; the **decoder** portion is used to train the whole model through language modeling.



[Raffel et al., 2018]

Pretraining on Encoder-Decoder

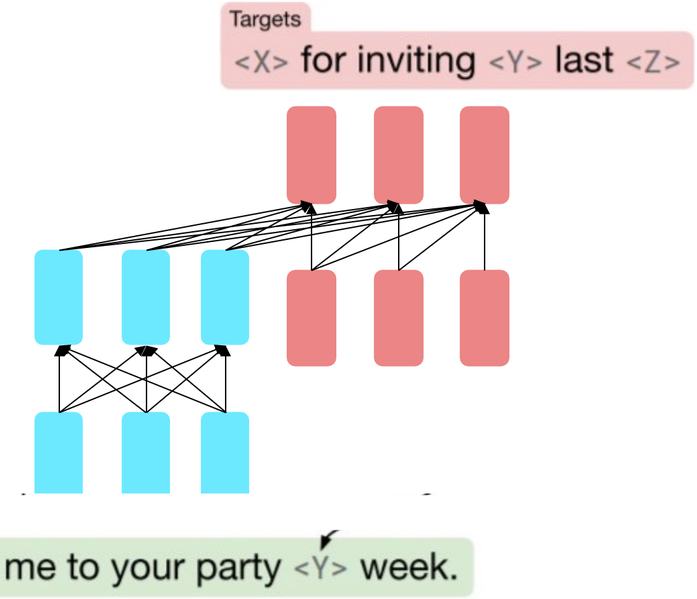
What Raffel et al., 2018 found to work best was **span corruption.** Their model: **T5**.

Replace different-length spans from the input with unique placeholders; decode out the spans that were removed!

Original text

Thank you ~~for inviting~~ me to your party ~~last~~ week.

This is implemented in text preprocessing: it's still an objective that looks like **language modeling** at the decoder side.



05. Applying BERT and GPT

Thank you.